



CPE 334 Introduction to Software Engineering in the Age of AI Agents

Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 4. TokTickIT Actions Taken, Dashboards, and Final Regression.

Score: ____ / 60

Instructors:	Assoc. Prof. Suthep Madarasma, Ph.D. (Jogie/โจ๊ก)	(suthep.mad@kmutt.ac.th)
	Aj. Piyanit Ua-areemitr, Ph.D (Toey)	(piyanit.wep@kmutt.ac.th)
	Aj. Santawat Thanyadit Ph.D. (Job)	(santawat.than@kmutt.ac.th)
TAs:	Rachawipa Katippatee (Bom)	(rachawipa.kati@gmail.com)
	Kantapat Suwannahong (Bump)	(kantapat.suwan@kmutt.ac.th)
	Rattanachote Petpansri (Loogmoo)	(rattanachote.petpa@kmutt.ac.th)
	Prapatsorn Sangrod (Noon)	(prapatsorn.sangr@kmutt.ac.th)
	Supachok Deetaweessukh (Tik)	(jedsadaporn.pann@mail.kmutt.ac.th)

1. Software Product Increment in Lab 4

Lab 4 completes the core TokTickIT service-desk workflow. It adds Actions Taken by IT Staff, enforces the final Ticket status rules, provides role-appropriate dashboards, and hardens the complete application built across Labs 1 to 3. By the end of this sprint:

- IT Staff and Administrators can create and update Actions Taken for a Ticket;
- a Ticket may have one primary Ticket Owner while different IT Staff members may take action and record in Actions Taken;
- Requesters, IT Staff, and Administrators receive role-appropriate dashboard information;
- the complete Ticket lifecycle is implemented and tested from creation through closure or cancellation;
- all previous authentication, authorization, Requester, IT Staff, Administrator, comment, note, and attachment behavior continues to work; and
- the application is polished, responsive, accessible, secure, and ready for final demonstration.

2. Lab 4 Learning Outcomes

- model and implement a parent-child work structure in which a Ticket has multiple Actions Taken lines;
- design and enforce business rules that span related records and state transitions;
- design dashboards that summarize operational data without replacing the underlying detailed views;
- extend an existing full-stack system while preserving regression behavior from earlier sprints;
- apply Spec DD, Test DD, and TDD to workflow, analytics, auditability, and final hardening;
- use GitHub Issues, feature branches, Pull Requests, peer review, and staged integration; and
- evaluate final product completion through traceable evidence rather than visual appearance alone.

3. Lab 4 Request from Stakeholder

“The service desk can now receive Tickets and IT Staff can communicate with Requesters, but we still need a reliable way to plan and track the actual work. Add Actions Taken under each Ticket. Each action should contain Action Date/Time, Action Description, Result, Performed by (auto), Follow-Up

Required?, Follow-up Note (required when follow-up is needed), Attachment Notes (what file to look for images etc.)

The primary Ticket Owner remains responsible for coordinating the Ticket as a whole. Requesters may continue to indicate that the problem appears resolved, but IT Staff must review the work and formally update the Ticket.

Add useful dashboards for Requesters and IT Staff, but keep them concise and connected to the detailed screens. Finally, polish and harden the complete application so that all earlier features continue to work consistently under the Zen Green design language.”

4. Engineering Contract for Sprint 4

Students must prepare and maintain the Sprint 4 engineering contract before implementation. It extends the earlier contracts and must define the Actions Taken model, dashboard calculations, API changes, UI behavior, regression coverage, and final Product Definition of Done.

4.1. What the Engineering Contract Must Cover

- Action Taken date/time, Action Description, Result, Performed by (auto), Follow-Up Required?, Follow-up Note (required when follow-up is needed), Attachment Notes (what file to look for images etc.)
- the complete Ticket status-transition matrix;
- Requester and IT Staff dashboard metrics, drill-down behavior, empty states, and calculation rules;
- database, Prisma migration, seed, and REST API increments;
- Zen Green UI extensions, responsive behavior, accessibility, and visual consistency;
- security, authorization, concurrency, safe failure, and regression behavior; and
- acceptance criteria, planned tests, final hardening evidence, and Product Definition of Done.

4.2. Explicitly Excluded from Lab 4

- automatic SLA clocks, escalation engines, on-call scheduling, and breach notifications;
- email, SMS, LINE, push, or other external notification services;
- inventory consumption, spare-parts management, purchasing, or cost accounting for services;
- time-sheet billing, payroll, or detailed labor-cost calculation;
- multi-level approval workflows and electronic signatures;
- advanced business-intelligence tools, custom report builders, or export warehouses;
- multi-tenant organizations and production-scale cloud operations; and
- new product features not approved in the Sprint 4 engineering contract.

4.3. Action Taken Roles and Responsibilities

Role	Minimum permitted behavior
Requester	View Actions Taken information for owned Tickets where approved by the specification. Requesters do not create or change Actions Taken.

IT Staff	Create and update Action Taken on accessible Tickets.
Administrator	Perform IT Staff behavior and retain administrative access needed for support and testing.

Students must complete the authorization matrix with the AI specification agent. Every write operation must be enforced by the backend. Hiding controls in the user interface is not authorization.

4.4. Required Action Taken Rules

The following are examples of mandatory rules, not the complete specification. Students must identify and number the remaining rules as BR-01, BR-02, and so on in docs/lab-04/specification.md.

BR ID	Example Mandatory Business Rule
BR-01	Action Taken belongs to exactly one Ticket.
BR-02	The Ticket Owner coordinates the Ticket, but an Action Taken may be by a different IT Staff member.

4.5. Ticket Status and Resolution Rules

- The Ticket statuses remain New, Open, In Progress, Waiting for Requester, Resolved, Closed, Reopened, and Cancelled.
- Students must refine and document the final permitted transition matrix and authorized roles.
- The backend must enforce the resolution rule even when a client bypasses the normal screen.
- A Requester indication that the problem appears resolved is advisory and does not itself change the Ticket to Resolved.

4.6. Dashboard Rules

Dashboards summarize existing operational data and provide links to detailed lists. Students must define exact metric names and calculations in the engineering contract. Examples include:

- Requester dashboard: total open Tickets, Tickets waiting for the Requester, recently updated Tickets, and recently resolved Tickets;
- IT Staff dashboard: unassigned Tickets, Tickets owned by the current user, Tickets by status or IT Priority, and recently updated Tickets; and
- Administrator dashboard: may reuse the IT Staff dashboard and optionally include concise user-account counts.

Metrics must be calculated by the backend from authoritative data. Every dashboard card or count must have a defined query, empty behavior, and drill-down destination where practical.

5. Required Database Increment

Students must evolve the existing PostgreSQL and Prisma design without discarding data from earlier labs. The design must support Actions Taken, dashboard queries, and any additional fields required by the approved Ticket workflow.

5.1. Required Concepts and Relationships

- one Ticket may contain many Action Taken;
 - all earlier Users, Tickets, Attachments, Public Comments, and Internal Notes remain valid after migration.
- Students must determine fields, data types, foreign keys, indexes, enums or reference tables, timestamps, optimistic-concurrency or stale-update handling, and migration strategy. At least two database-design decisions must be justified in specification.md.

5.2. Required Migration and Backfill

The migration must preserve all existing data. Students must define how legacy Tickets without Actions Taken behave, and how dashboard calculations treat existing records. The migration and rollback or recovery approach must be documented and tested.

5.3. Required Seed Data

- idempotent seed behavior that is safe to run repeatedly;
- realistic Tickets covering all major statuses, priorities, assigned and unassigned ownership;
- Tickets with zero, one, and multiple Actions Taken; and
- data sufficient to demonstrate non-zero and zero dashboard metrics.

6. Required REST API Contract

The REST API must support the capabilities below. Students must define exact endpoint paths, methods, request and response shapes, validation, authorization, safe errors, conflict handling, and status codes in docs/lab-04/api-spec.md.

- create and update Actions Taken as permitted;
- retrieve Requester dashboard data;
- retrieve IT Staff dashboard data;
- continue all approved APIs from Labs 2 and 3; and
- support final health, validation, and regression behavior required by the contract.

6.1. Ticket Resolution and Conflict Behavior

The API contract must define how concurrent or stale updates are detected or safely handled so that one user does not unknowingly overwrite another user's recent workflow change.

6.2. Dashboard Contract

Dashboard endpoints must return concise, documented data rather than entire Ticket collections. Students must define the calculation for each metric, the time zone and date boundaries where relevant, links or query parameters used for drill-down, and behavior when no matching records exist.

7. Zen Green Theme and Final Application Shell

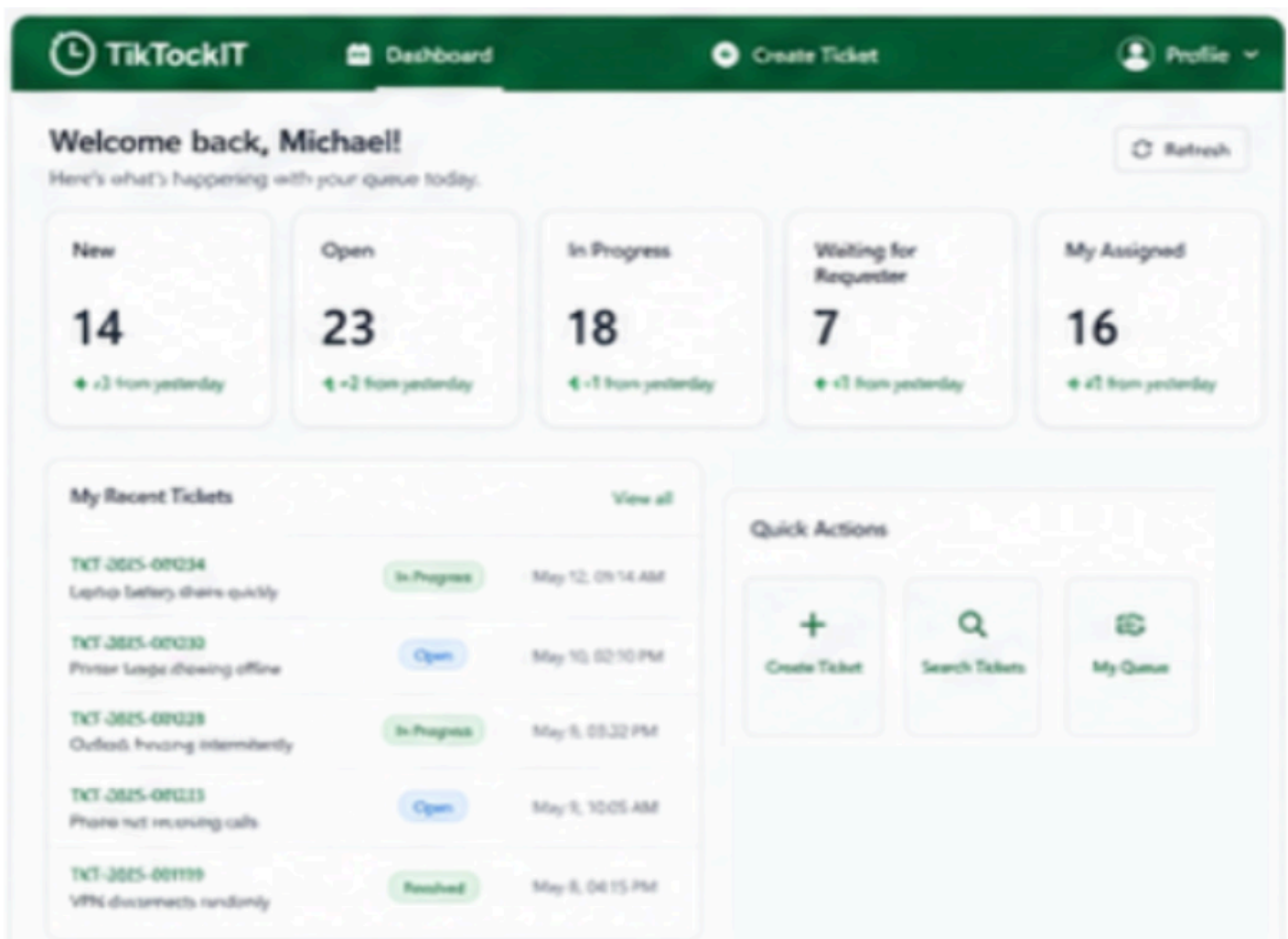
Lab 4 must preserve the Zen Green design language established in Lab 2 and extended in Lab 3. The final product should look and behave like one coherent application.

- add role-appropriate Dashboard navigation and maintain clear active-page indication;
- reuse existing Ticket, tab, badge, button, form, table, card, loading, empty, error, and responsive conventions;
- visually distinguish Ticket status, priorities, and private versus shared content;
- use concise metric cards with labels, values, and accessible drill-down actions;
- preserve visible focus, keyboard operation, semantic labels, and non-color status cues;
- avoid clipped content, overlapping controls, inaccessible modal dialogs, and horizontal page scrolling; and
- remove temporary, duplicate, obsolete, or inconsistent UI elements left from earlier labs.

8. Required User Interfaces

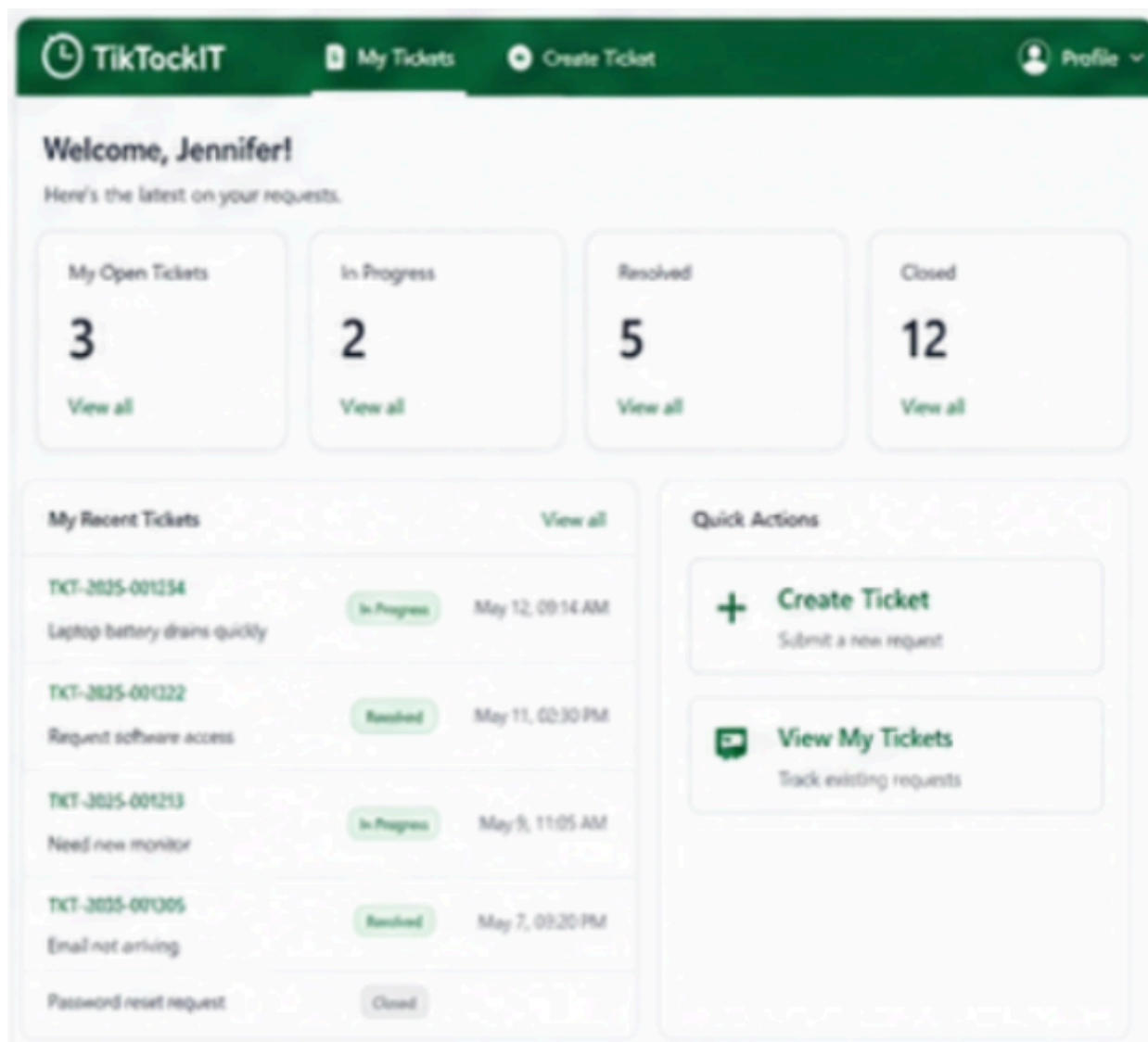
8.1. IT Staff Dashboard

The IT Staff Dashboard must provide a concise operational starting point. It must show approved metrics and recent or urgent Ticket information. Each actionable item should open the appropriate Ticket Queue, Ticket Detail, or filtered view. Students must define the final cards, lists, responsive arrangement, loading, empty, forbidden, and safe-failure feedback in [ui-spec.md](#).



8.2. Requester Dashboard

The Requester Dashboard must summarize only the authenticated Requester's Tickets. It should help the Requester identify Tickets requiring attention, recently updated Tickets, and resolved work without duplicating the full My Tickets screen. Ownership protection must remain enforced by the backend.



8.3. Actions Taken on Ticket Detail

The existing IT Staff Ticket Detail screen must add an Actions Taken area. It must support an appropriate list or table, create mode and view/edit mode. Requesters will see all Actions Taken items. Each Actions Taken will have Action create date/time, Action Description, Result, Performed by (auto), Follow-Up Required?, Follow-up Note (required when follow-up is needed), Attachment Notes (what file to look for images etc.)

8.4. Ticket Workflow and Resolution Feedback

Ticket status controls must show only permitted transitions. The user interface should guide the user, but the backend remains the authority. Successful changes must refresh the Ticket summary status.

8.5. Final Regression and Product Hardening

- all Requester, IT Staff, and Administrator screens from earlier labs remain available to permitted users;
- all role navigation, ownership, comments, notes, attachments, user management, and authentication behavior remains correct;
- loading, validation, success, empty/no-results, forbidden, conflict, not-found, and safe API-failure feedback is consistent;
- duplicate actions caused by repeated clicking or network retry are prevented or safely handled;
- important forms protect entered data after recoverable failures;
- console errors, broken links, placeholder text, and unfinished controls are removed; and
- README setup, seed, migration, test, and demonstration instructions are current.

8.6. Responsive and Accessibility Requirements

- Same as Lab 2 and 3.

9. Spec DD Deliverable

Required files
docs/lab-04/specification.md
docs/lab-04/ui-spec.md
docs/lab-04/api-spec.md

Students must transform this handout into a concise and internally consistent Sprint 4 engineering specification. Do not copy the entire handout. Resolve implementation choices, identify assumptions, and explain how earlier increments are preserved.

Section	What the Student Must Provide
1. Sprint Goal	One short paragraph stating the delivered value.
2. Stakeholder Request	A concise interpretation in the student's own words.
3. Scope	Included and explicitly excluded work.
4. Functional Requirements	Numbered FR statements for Actions Taken, workflow, dashboards, and hardening.
5. Business Rules	Numbered BR statements including assignment, dates, statuses, resolution gate, and dashboard calculations.
6. UI Specification Summary	Screen structure, modes, controls, feedback, role behavior, responsive rules, and reference to ui-spec.md.
7. Data Changes	Models, fields, relationships, indexes, migration, backfill, and seed decisions.
8. API Contract	Endpoints, request/response shapes, statuses, authorization, conflicts, and safe errors.
9. Acceptance Criteria	Observable and testable criteria such as AC-01.

10. Definition of Done	Product-completion checklist used by the coding agent.
11. Assumptions and Decisions	Only meaningful choices not fixed by the handout.

9.1. Example Acceptance Criteria

ID	Example Criterion
AC-01	Given a permitted IT Staff user and valid data, when an Actions Taken is created, then it is saved under the correct Ticket with the authenticated creator and approved assignee.
AC-02	Given an authenticated Requester, when dashboard data is retrieved, then only metrics and recent Tickets owned by that Requester are returned.

Students must add enough criteria to cover the complete approved scope. Every Acceptance Criterion must map to at least one planned test.

10. Test DD and TDD Deliverable

Required file
docs/lab-04/tests.md

The test plan must be created before or alongside implementation. It must include unit, API or integration, UI component, UI style, responsive, authorization, workflow, migration/regression, performance-smoke, and end-to-end coverage.

Test ID	Type	Requirement / AC	What It Tests	Expected Result	Automated Test File	Final
API-03	API	AC-01	Create a valid Actions Taken	Created under the correct Ticket and actor	server/tests/lab-04/actions-taken.api.test.ts	Pass
E2E-02	E2E	AC-03				Pass

Students must identify the remaining tests for all UI, dashboard calculations, drill-down, role restrictions, concurrent or stale updates, safe failures, responsive behavior, accessibility, and complete Labs 1 to 3 regression.

11. GitHub Issues and Workflow

Use the same Kanban statuses introduced earlier. Before coding, decompose Sprint 4 into a reasonable set of GitHub Issues covering specification, tests, migration, Actions Taken, Ticket workflow, dashboards, regression, accessibility, visual inspection, and release integration.

Example Issue	Possible Scope
---------------	----------------

Sprint 4 engineering contract	specification.md, tests.md, ui-spec.md, and api-spec.md.
Actions Taken foundation	Prisma migration, model, seed, APIs, authorization, and tests.
Actions Taken UI	Ticket Detail list, create/edit behavior, responsive layout, and tests.
Ticket workflow	Resolution gate, status transitions, UI, and tests.
Role dashboards	Requester and IT Staff dashboard APIs, metrics, drill-down, UI, and tests.
Final hardening	Regression, accessibility, visual consistency, error handling, and release verification.

11.1. Required Branch Flow

- Similar to Labs 2 and 3.

11.2. AI Specification Agent and Coding Agent Rules

- Similar to Labs 2 and 3.

12. Required Repository Increment

Minimum Lab 4 structure

docs/lab-04/
├── specification.md
├── tests.md
├── ui-spec.md
├── api-spec.md
├── reviewer.md
└── ai-use.md
server/tests/lab-04/
├── actions-taken.api.test.ts
├── ticket-workflow.api.test.ts
├── requester-dashboard.api.test.ts
└── staff-dashboard.api.test.ts
client/.../lab-04 tests/
├── StaffDashboard.test.tsx
├── RequesterDashboard.test.tsx
├── ActionsTaken.test.tsx
└── TicketWorkflow.test.tsx
e2e/lab-04/
├── actions-taken-flow.spec.ts
└── ticket-resolution.spec.ts

```
└─ dashboards.spec.ts

artifacts/lab-04/screenshots/
├─ staff-dashboard/
├─ requester-dashboard/
└─ actions-taken/
```

13. Definition of Done for Lab 4

- Similar to Labs 2 and 3. Students are expected to work with an LLM to finalize the Definition of Done for Product Completion.

14. Submit One PDF File

Submit exactly one concise PDF. To make grading consistent for approximately 200 students, use the headings “Answer Part 1” through “Answer Part 9” in this exact order. Include working links. Screenshots must be readable without extreme zoom. The submitted repository and final main branch remain the source of truth.

Part	Points	Required Submission Evidence
1. Git Use with Engineering Workflow	10	Commit-history evidence showing feature branches merged into lab4-staging and then main; final GitHub Project/Kanban with all Issues in Done; rendered reviewer.md with reviewer identity, PR links, comments, responses, and approvals; README and .gitignore evidence; repository directory structure.
2. Spec DD	5	Link to and rendered docs/lab-04/specification.md. Show numbered requirements, business rules, Actions Taken and Ticket transition rules, dashboard calculations, acceptance criteria, migration decisions, and Product Definition of Done. Include evidence that the specification existed before the main implementation PRs were completed.
3. Test DD and Traceability	10	Link to and rendered docs/lab-04/tests.md. Include planned tests, Acceptance-Criterion traceability, actual test-file paths, and final status. Include complete unit, API/integration, UI, authorization, workflow, regression, and E2E passing test output from main.
4. AI Use with Reflection	5	Rendered docs/lab-04/ai-use.md naming the LLM used and showing 6-10 selected key prompts. Provide a brief “My Reflection” on specification-agent and coding-agent use.
5. Working IT Staff Dashboard UI	5	Demonstrate approved operational metrics, current-user Actions Taken, recent or urgent Tickets, accurate counts, drill-down behavior, loading, empty, forbidden, safe-failure, and responsive behavior. Include evidence that selected metrics match database queries.

6. Working Actions Taken UI	10	Demonstrate list, create, assign, edit, status transition, complete, cancel, validation, inactive-assignee rejection, role restrictions, safe failures, and responsive behavior. Show different Actions Taken on one Ticket.
7. Working Ticket Workflow	5	Demonstrate permitted Ticket transitions, stable ordering, append-only behavior, and role-appropriate visibility.
8. Working Requester Dashboard and Final Regression UI	5	Demonstrate Requester-owned metrics, recent and attention-required Tickets, drill-down, ownership protection, and representative regression evidence for authentication, My Tickets, Ticket Detail, Attachments, Public Comments, IT Staff functions, Internal Notes, and Administrator user management.
9. Zen Green UI, Responsive, Accessibility, and Final Polish	5	Rendered ui-spec.md plus desktop, tablet, and mobile screenshots for all major Lab 4 screens. Include the completed visual and accessibility checklist for design consistency, dashboards, Actions Taken, editable/read-only fields, validation placement, keyboard focus, clipping, overlap, and horizontal overflow.

To make grading faster and more consistent, format the PDF using:

Answer Part 1:

[Place your content here]

Answer Part 2:

[Place your content here]

...

Answer Part 9:

[Place your content here]